



Chapter 6

Working with a NoSQL DMBS

IN THIS CHAPTER

» Understanding hierarchical data

» Using hierarchical datasets

» Considering other forms of hierarchical data

.....

The previous chapter talks about the inordinate amount of organization surrounding Relational Database Management Systems (RDBMSs). In fact, authors write huge tomes about the topic, yet no one resource manages to communicate just how organized these databases are. The hierarchical database is quite the opposite in terms of organization. It isn't untidy by any means; it always knows which pile contains the data it has on hand. Still, sometimes the viewer wonders about it all at first glance. Yet, you often need a hierarchical form of data storage, such as the Not Only SQL (NoSQL) database, to store the chaotic information of the real world in a manner that would elude the RDBMS. That type of storage capability is what this chapter considers.

[Books 3](#) and [4](#) of this minibook give you the underpinnings and simplified techniques of hierarchical data storage in the form of trees, graphs, and XML. You could add JavaScript Object Notation (JSON) (see [Chapter 1](#) of this minibook for an overview of JSON as a raw data storage methodology) into the mix of technologies that allow hierarchical

storage as well. The first part of this chapter takes a more detailed look at the various strategies used to store data in a hierarchical form.

The next part of the chapter discusses how data access occurs. You spend some time looking at a NoSQL implementation and discover how you access it as a database engine using techniques similar to those used with an RDBMS. Even though the underlying storage strategies differ greatly from an RDBMS, any useful hierarchical data storage methodology must provide a way to manage records in a consistent manner.

The final part of this chapter discusses some of the alternative forms of hierarchical data storage. All the straightforward methods of data storage might seem to be taken already, but that's not the case. As long as humans have a need to store data, someone will come up with another method of doing so that provides some benefit not offered by the other methods.

Considering the Ramifications of Hierarchical Data

You can use hierarchies to store data of various types. A hierarchy has the advantage of providing a searchable method of locating disorganized data. Each data element may differ in size and content, but the hierarchy will provide a means of searching it anyway. This ability to locate what you might not otherwise find in an RDBMS is a significant difference between the RDBMS and the hierarchy, as explained in the following sections.

Understanding hierarchical organization

Hierarchies appear all over the place. The organizational chart used to describe the responsibilities, status, and interrelationships in a company is a hierarchy. Deep learning requires the use of hierarchies of neurons to perform tasks. Trees and graphs are programming struc-

tures that rely on hierarchies. In fact, if you really think about it, you can't get around the hierarchy. Even looking outside your window, you see plants, shrubs, bushes, and yes, even trees that form hierarchies. Hierarchies of animals also exist when it comes to the food chain, with the most capable predators at the top and the least aware microscopic organisms at the bottom. Sometimes the linkage in a hierarchy seems to defy imagination, such as with the whale that eats massive quantities of krill and plankton. A hierarchy can happen anywhere, even in completely unexpected places and in the most difficult to understand ways.

For all the complexities that hierarchies represent, however, they're actually quite simple in construction. A hierarchy normally consists of these elements:

- Object nodes, whether real, imagined, or abstract in nature
- Linkages between nodes
- Directionality, whether top to bottom, one-way, two-way, cyclic, or any other direction you can imagine
- Attributes that describe node, linkage, and directionality as needed (or are absent when not)



REMEMBER In some minds, a hierarchy denotes a system of ranking objects, as in an organizational chart. However, you can use the hierarchical structure without ranking anything. For example, when you use a hierarchy to define the content of a warehouse, the hierarchy doesn't rank the various items; rather, it might simply group like items together under the root node of the warehouse as a whole.

Developing strategies for freeform data

You can easily assign strategies that are normally employed by RDBMS to hierarchical databases when the only experience a person has is with the RDBMS. For example, when working with a warehouse, the

automatic response to linkage is to attach a part number to everything. Unfortunately, although a part number works fine in an RDBMS, it doesn't work at all with a hierarchy.

CHOOSING HIERARCHIES OVER RDBMS

An RDBMS works best when data is consistent, records have approximately the same size, and you have a unique method of identifying each item. For example, if your goal is to catalog a warehouse strictly according to part number, product name, location, and so on, then an RDBMS will work best because each item is formatted precisely the same. The problem is that this form might not serve well when you have a warehouse containing a large group of unlike items.

Another way to view the warehouse is as a group of objects with unique characteristics, such as whether the item is frozen or refrigerated when such a designation works. Some items may have colors; others may employ specific metals. In this case, you work with items based on individual attributes rather than rely on some other strategy. A hierarchical organization works best in this case because you need to group like items together. For example, you wouldn't put metal working equipment and food items in the same group; they're different sorts of items based on how you interact with them. Likewise, you might put the blue metal working equipment into one group and the red metal working equipment into another group, with the difference being the color in this case.



REMEMBER What you really want is something that describes the item, such as the item name. For example, item number 1234-5678JX isn't descriptive, but Radial Tires (P 225/70R14 98S SL RWL) is. Making the name descriptive is helpful in differentiating like items. You may

have many different radial tire sizes, so providing a descriptive part number will help.

Of course, using a consistent numbering strategy is relatively easy to search, as long as you know the item number. Searching a hierarchy of names is harder unless you use techniques such as those found in Book 4, [Chapter 4](#) to perform natural language processing. By using the correct search techniques, you can get around issues like

- Misspellings
- Word order
- Nomenclature consistency
- Missing terms
- Punctuation
- Diacritical marks
- Language inconsistencies (think *colour* versus *color*)

In addition, you may find performing tasks such as displaying the item data more difficult because each item can contain unique information. Creating flexible forms is an essential element of making freeform data stored in a hierarchy manageable. The form will need to construct its content based on the actual item data available.

The point is that freeform data requires a different kind of handling than the rows and columns found in an RDBMS table. You may even find it necessary to create methods for restructuring the hierarchy at times. Perhaps you normally link items together using the item name, but you find a sudden need to link items by physical location instead.

Performing an analysis

Most data scientists don't have a problem figuring out how to perform an analysis using RDBMS data because the data is already in tabular form, which is compatible with programming structures like a pandas `DataFrame`. Freeform data presents certain challenges based on the need to describe an item, rather than simply document it (as is done in

an RDBMS). You may need to squint at the data a little and choose a different means of obtaining analysis information based on the following:

- **Consistent attributes:** Even if the attributes vary greatly among items, some attributes may remain consistent, such as size, value, weight, and so on. You can use these consistent elements as a basis for comparison, such as value and size versus storage cost.
- **Item inconsistencies:** Given enough items in a particular group, you may be able to perform an analysis of items in that group (rather than the warehouse as a whole) based on item inconsistencies.
- **Promised versus realized attributes:** The value of an item description is that you can determine whether the promised item composition matches the item as it actually appears in the real world.
- **Missingness:** Items in one group may have attributes or attribute values that are missing in another, similar, group.



TIP

Generally, you can't use every part of an item description in an analysis, just as you wouldn't use the information in the real world. For example, when shopping at the store, the cost and perceived value of an item are more important than the color of the packaging, even though the color of the packaging may ultimately affect whether you buy the item. Packaging can also affect issues such as how many items you buy. However, when you view two like items and one is significantly more expensive than the other, you're more likely to let price decide which item you get.

Freeform data arranged in a hierarchy focuses on description, so your analysis must focus on description, too. You must think through the process of determining which attributes are consistent enough to allow for the particular analysis you need to perform. You need to consider how a descriptive entry can make your analysis more efficient or correct than using the abstractions of an RDBMS.

Working around dangling data

The one area in which an RDBMS and a hierarchical data source have an equal set of problems is with dangling data — the orphaned record or object that simply doesn't connect to anything. With an RDBMS, the problem generally occurs as the result of code that deletes a parent record without also removing all the child records. Because of the nature of an RDBMS, locating such records or eliminating them from happening at all is relatively easy using rules. Dangling freeform data in a hierarchical form isn't nearly so easy to locate.

The problem is one of linkages. It's hard to tell sometimes whether every object in the database connects to something else (ending with the root item that connects to everything else). The only way to locate dangling data is to traverse the tree as you would a graph, binary search tree (BST), or binary heap. For example, consider [Figure 6-1](#). If Leaf F is disconnected from the rest of the tree, you need to

1. Determine that Leaf F exists.
2. Traverse the entire tree to know what is and isn't connected (creating a list of nodes in the process).
3. Search for Leaf F in the connection list.
4. Define Leaf F as disconnected when it doesn't appear in the list.

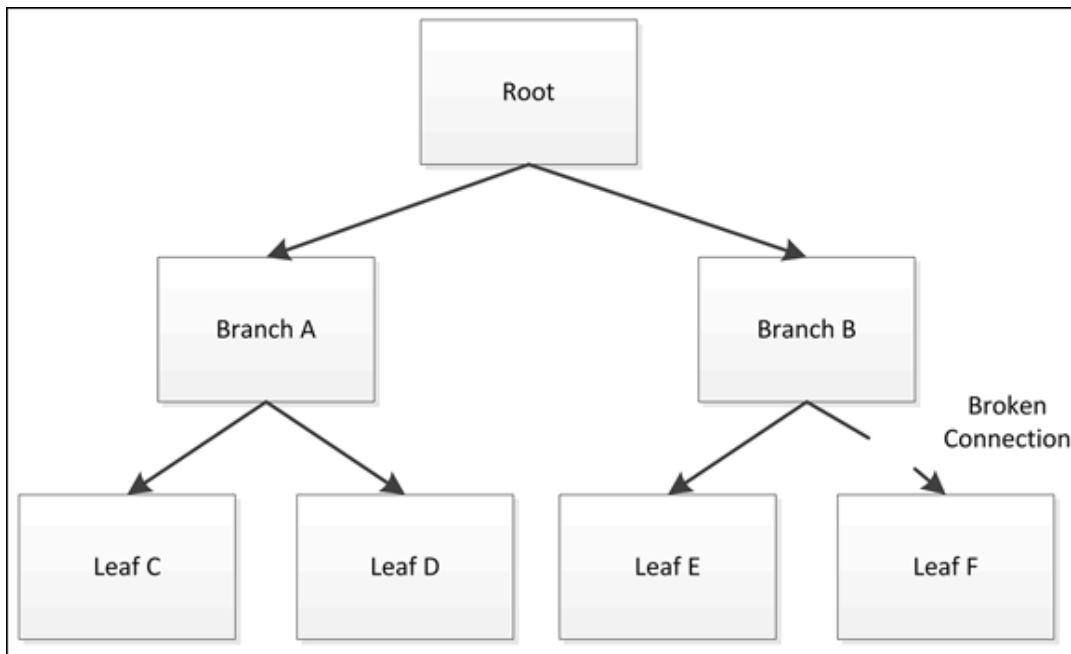


FIGURE 6-1: A hierarchical construction relies on links to each item.

The process of knowing that Leaf F exists is problematic. You could create a historical database to track which items should appear in the database at any given time. The process is incredibly time consuming and error prone, yet you must have a procedure in place for locating these dangling data items or database corruption will quickly create issues for any analysis you want to perform. Unfortunately, the tree support for Python is lacking, and you sometimes can't locate a library to perform the task, either, so some of these error-handling techniques require custom code. Chapters 5 through 7 of *Algorithms For Dummies*, by John Paul Mueller and Luca Massaron (Wiley), provide code and extensive details about how to manage various kinds of tree structures.

CONSIDERING THE TYPES OF TREES

Different kinds of data storage structures are available. Here is a quick list of the kinds of structures you commonly find:

- **Balanced trees:** A kind of tree that maintains a balanced structure through reorganization so that it can provide reduced access times. The number of elements on the left side differs from the number on the right side by one at most.

- **Unbalanced trees:** A tree that places new data items wherever necessary in the tree without regard to balance. This method of adding items makes building the tree faster but reduces access speed when searching or sorting.
 - **Heaps:** A sophisticated tree that allows data insertions into the tree structure. The use of data insertion makes sorting faster. You can further classify these trees as max heaps and min heaps, depending on the tree's capability to immediately provide the maximum or minimum value present in the tree.
-

Accessing the Data

Any data you store is useless unless you can obtain access to it. How this access occurs depends on the data storage strategy:

- Flat-file storage such as XML requires a search of the tree.
- In-memory structures, such as dictionaries, rely on keys that you can locate using a single search.
- NoSQL databases rely on keys as well, but you can also sort and index them in various ways.

You need to be aware of the kind of data storage that you're using, including its limits and advantages. The following sections provide an overview of data access in a general way, discussing some of the ways in which hierarchical data storage methods can differ.

Creating a picture of the data form

[Figure 6-1](#) shows a basic hierarchy and how you start at the root node to find any particular leaf. The picture shows a basic binary tree. Not every form of hierarchical storage works this way, though. You may find that the data storage you use relies on any of these structures:

- **Ternary tree:** Uses up to three links for each data node.
- **Binary Search Tree (BST):** The left and right subtrees must also be BST.
- **Adelson-Velsky and Landis (AVL) tree:** A kind of self-balancing BST, which improves tree performance.

Many other kinds of trees exist as well, and sometimes a data storage technique will rely on a graph instead. Without a picture of what the data looks like, you have no hope of truly understanding how to access any particular node. Consequently, one of the first issues you must deal with is determining how the data storage is laid out, and the best way to do this is in graphic form. The article at

<https://www.freecodecamp.org/news/all-you-need-to-know-about-tree-data-structures-bceacb85490c/> tells how someone actually draws the tree structure to better understand it.

Employing the correct transiting strategy

Of all the tasks that applications do, searching is the more time consuming and also the one required most. Even though adding data (and sorting it later) does require some amount of time, the benefit of creating and maintaining a dataset comes from using it to perform useful work, which means searching it for important information.

Consequently, you can sometimes get by with less efficient Create, Read, Update, and Delete (CRUD) functionality and even a less-than-optimal sort routine, but searches must proceed as efficiently as possible. The only problem is that no one search performs every task with absolute efficiency, so you must weigh your options based on what you expect to do as part of the search routines.

Two of the more efficient methods of searching involve the use of the binary search tree (BST) and binary heap. Both of the search techniques rely on a tree-like structure to hold the keys used to access data elements. However, the arrangement of the two methods is different, which is why one has advantages over the other when performing certain tasks. [Figure 6-2](#) shows the arrangement for a BST.

Note how the keys follow an order in which lesser numbers appear to the left and greater numbers appear to the right. The root node contains a value that is in the middle of the range of keys, giving the BST

an easily understood balanced approach to storing the keys. Contrast this arrangement to the binary heap shown in [Figure 6-3](#).

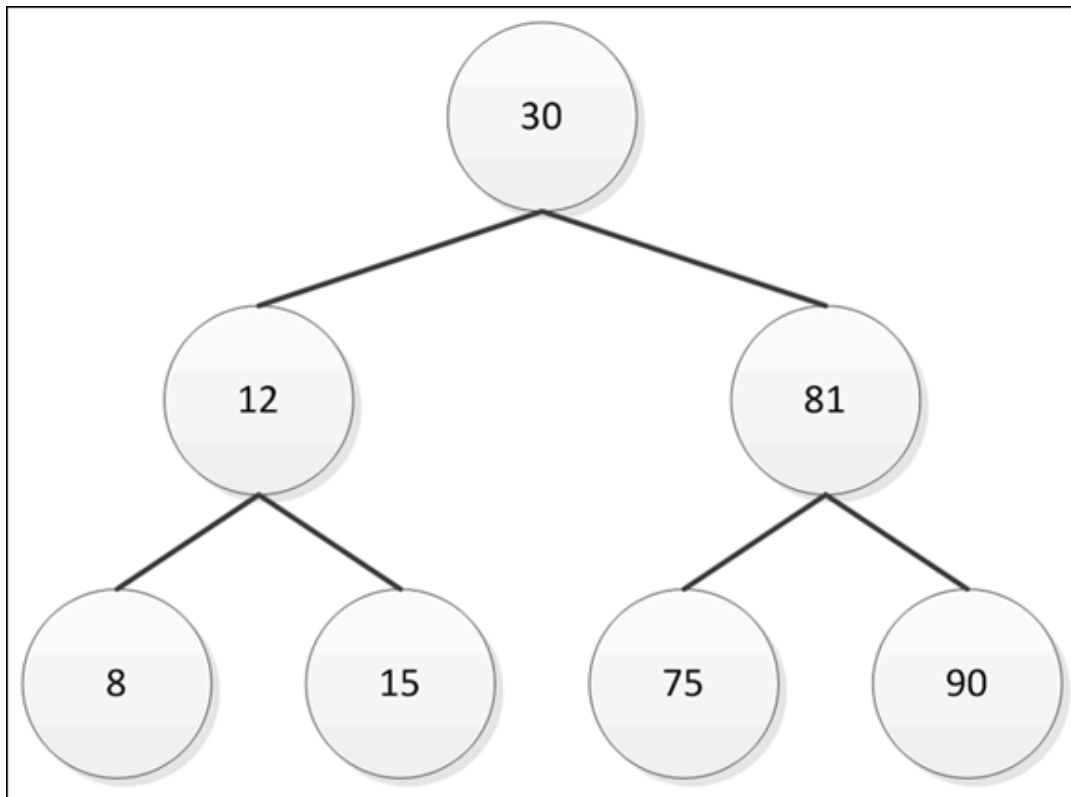


FIGURE 6-2: The arrangement of keys when using a BST.

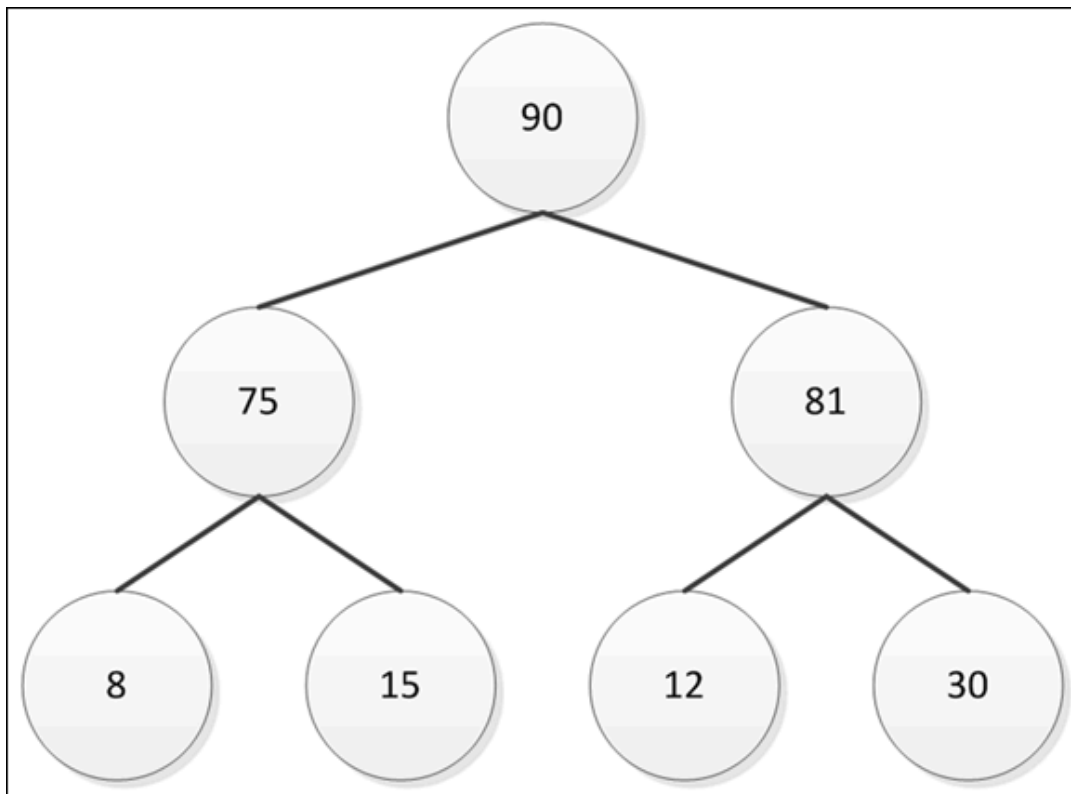


FIGURE 6-3: The arrangement of keys when using a binary heap.

Each level contains values that are less than the previous level, and the root contains the maximum key value for the tree. In addition, in this particular case, the lesser values appear on the left and the greater on the right (although this order isn't strictly enforced). The figure actually depicts a *binary max heap*. You can also create a *binary min heap*, in which the root contains the lowest key value and each level builds to higher values, with the highest values appearing as part of the leaves.

The manner you use to traverse a tree or graph is important. Consider the graph shown in [Figure 6-4](#) as the starting point for discussion in this case.

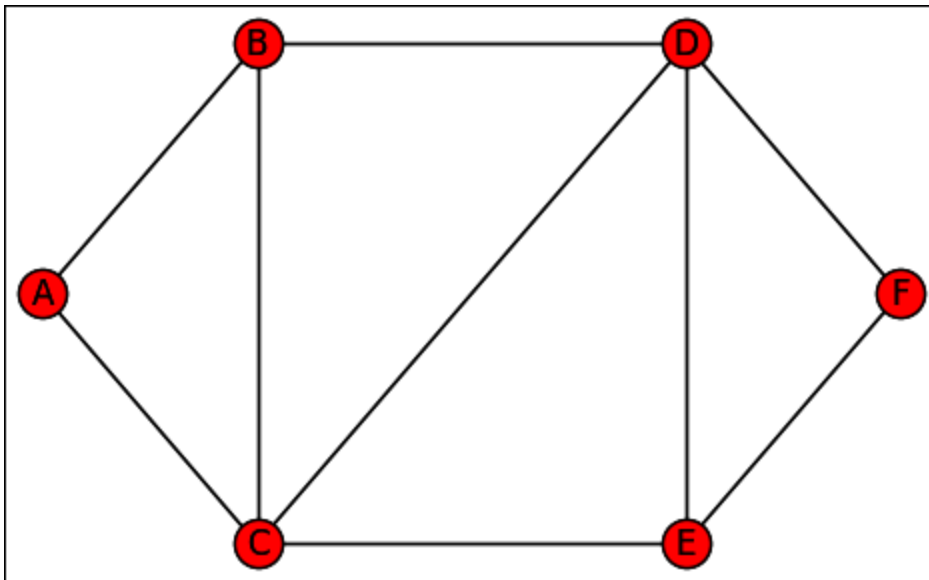


FIGURE 6-4: An example graph that you can use for certain types of data storage.

The most popular techniques in this case are the Breadth First Search (BFS) and Depth First Search (DFS):

- **BFS:** Begins at the graph root and explores every node that attaches to the root. It then searches the next level — exploring each level in turn until it reaches the end. Consequently, in the example graph, the search explores from A to B and C before it moves on to explore D. BFS explores the graph in a systematic way, exploring vertexes all around the starting vertex in a cir-

cular fashion. It begins by visiting all the vertexes a single step from the starting vertex; it then moves two steps out, then three steps out, and so on.

- **DFS:** The algorithm begins at the graph root and then explores every node from that root down a single path to the end. It then backtracks and begins exploring the paths not taken in the current search path until it reaches the root again. At that point, if other paths to take from the root are available, the algorithm chooses one and begins the same search again. The idea is to explore each path completely before exploring any other path. To make this search technique work, the algorithm must mark each vertex it visits. In this way, it knows which vertexes require a visit and can determine which path to take next.

Even though [Figure 6-4](#) shows a graph, the same techniques work with trees. The article at <https://www.geeksforgeeks.org/bfs-vs-dfs-binary-tree/> describes the same process using a tree rather than a graph. Using BFS or DFS can make a difference according to the way in which you need to traverse a graph. From a programming point of view, the difference between the two algorithms is how each one stores the vertexes to explore the following:

- A queue for BFS, a list that works according to the FIFO principle. Newly discovered vertexes don't wait long for processing.
- A stack for DFS, a list that works according to the last in/first out (LIFO) principle.

The choice between BFS and DFS depends on how you plan to apply the output from the search. Developers often employ BFS to locate the shortest route between two points as quickly as possible. This means that you commonly find BFS used in applications such as GPS, where finding the shortest route is paramount. For the purposes of this book, you also see BFS used for spanning tree, shortest path, and many other minimization algorithms.

A DFS focuses on finding an entire path before exploring any other path. You use it when you need to search in detail rather than generally. For this reason, you often see DFS used in games, in which finding

a complete path is important. It's also an optimal approach to performing tasks such as finding a solution to a maze.



REMEMBER Sometimes you have to decide between BFS and DFS based on the limitations of each technique. BFS needs lots of memory because it systematically stores all the paths before finding a solution. On the other hand, DFS needs less memory, but you have no guarantee that it'll find the shortest and most direct solution.

Ordering the data

You have essentially two methods of ordering your hierarchical data: sorting and indexing. When sorting the data, you create a new version of the dataset in a specific order depending on the storage strategy you use. You use sorting only on small datasets. Indexing is the method of choice when working with large datasets. Even though you have a lot of options when it comes to indexing, here are the most popular:

- **B-Tree:** This is one of the most common indexing methods for DBMSs. When working with a B-tree, the internal nodes can have a variable number of child nodes depending on a predefined range. The advantage of this approach is that the code has to balance the tree less often. The disadvantage is that the tree has more unused space.
- **B+-Tree:** This is a variation of the B-tree in which all the keys reside in the leaf nodes, providing a boost in speed.
- **T-Tree:** This is a mix of AVL-trees (see <https://www.geeksforgeeks.org/avl-tree-set-1-insertion/> for details) and B-trees in which the resulting tree is balanced but each node can have a different number of children. A T-Tree tends to have better overall performance than a corresponding AVL-Tree. The nodes each contain one key/value pair and a pointer tuple. A T-Tree provides three kinds of nodes:
 - **T-Node:** Has both a right and left child
 - **Half-leaf node:** Has just one child
 - **Leaf node:** Has no children

- **O2-Tree:** This is an evolution of Red-Black trees (see <https://www.geeksforgeeks.org/red-black-tree-set-1-introduction-2/> for details), which is a form of Binary-Search tree in which the leaf nodes contain a tuple with the key/value pairs and a pointer to the next node. The basic reason to use this kind of indexing is that it enhances overall indexing performance. As with Red-Black trees, every node is either red or black, with a black root. When a node is red, both of its children are black. Leaf nodes are double linked both forward and backward, making this index incredibly easy to traverse.

Interacting with Data from NoSQL Databases

Hierarchical data formats can take many forms. For example, XML and JSON files are a type of hierarchical storage. However, these flat file formats are relatively limited in scope, so you need something with additional functionality, such as the databases described in the slide show at

<https://www.computerworld.com/article/3412345/the-best-nosql-database-options-for-the-enterprise.html>. Many of these offerings are in the cloud, so you don't even have to worry about creating a localized implementation. All these NoSQL databases are designed to handle the complexities of corporate data in a manner consistent with hierarchical storage.

NoSQL databases are used in large data storage scenarios in which the relational model can become overly complex or can break down in other ways. The databases generally don't use the relational model. Of course, you find fewer of these DBMSs used in the corporate environment because they require special handling and training. Still, some common DBMSs are used because they provide special functionality or meet unique requirements. The process is essentially the same for using NoSQL databases as it is for relational databases:

1. Import required database engine functionality.
2. Create a database engine.

3. Make any required queries using the database engine and the functionality supported by the DBMS.

The details vary quite a bit, and you need to know which library to use with your particular database product. For example, when working with MongoDB (<https://www.mongodb.org/>), you must obtain a copy of the PyMongo library

(<https://api.mongodb.org/python/current/>) and use the `MongoClient` class to create the required engine. The MongoDB engine relies heavily on the `find()` function to locate data. Following is a pseudo-code example of a MongoDB session. (You won't be able to execute this code in Notebook; it's shown only as an example.)

```
import pymongo
```

```
import pandas as pd
```

```
from pymongo import Connection
```

```
connection = Connection()
```

```
db = connection.database_name
```

```
input_data = db.collection_name
```

```
data = pd.DataFrame(list(input_data.find()))
```

Working with Dictionaries

[Chapter 2](#) of this minibook describes how to create and use dictionaries. In that chapter, you use scalar values as the value portion of the key/value pair. However, you don't have to use a scalar value. You can use other standard data types as the value, or you can even create a class to hold custom data. The point is that you use the key to locate the data and the value to hold the data.

Here's an example of using a list. You can create a dictionary containing a series of lists like this:

```
X = {'a': [1, 2, 3], 'b': [4, 5, 6]}
```

To access a particular key, you use code like this:

```
print(X['a'])
```

The output in this instance is a list: `[1, 2, 3]`. To access a given list element, you use code like this:

```
print(X['a'][1])
```

What you see is the value `2` as output. The point of this example is that you could easily implement a hierarchical data storage setup using a dictionary by providing a pointer to the next key as one of the value elements, like this:

```
X = {'a': ['b', 1, 2, 3], 'b': ['a', 4, 5, 6]}
```

To move to the next node, you simply index it as you would a value:

```
print("Next element is: ", X['a'][0])
```

You see `Next element is: b` as the output. You can create a B-tree or any other hierarchical structure using this approach if desired. Using a custom class is similar to working with a list, but you have to design the class. The article at <https://code.tutsplus.com/tutorials/how-to-implement-your-own-data-structure-in-python--cms-28723> offers a discussion of this approach.

Developing Datasets from Hierarchical Data

You can work with hierarchical data directly in your applications. It means using various traversal techniques and writing a lot of code to

deal with the various nodes. Unfortunately, it also means that you have to reinvent the wheel in performing many common tasks that libraries like pandas make extremely easy. Still, sometimes you have no other choice but to work with the data in hierarchical form.

Sometimes, you may find that your hierarchical data is amenable to being stuffed into a pandas `DataFrame` by using smart coding techniques and looking for patterns. For example, the discussion at <https://stackoverflow.com/questions/48374823/parsing-hierarchical-data-from-xml-to-pandas> talks about moving a particular XML dataset (which is hierarchical) into a `DataFrame`. In looking at the data for this example, you see that it's already in a somewhat tabular format. If the data were completely freeform, the example wouldn't work because creating the required rows and columns wouldn't be possible. Even so, this XML example is somewhat typical of certain kinds of data, such as log entries.



WARNING You can find some examples of complex data imports online, but generally the code for these examples is extremely brittle, which means that it'll break the moment anything unexpected happens with your data source. In many cases, unless the data you want to use as part of a `DataFrame` is already in a form that lends itself to tabular display, the better bet is to simply work with it as a hierarchical data source.

Processing Hierarchical Data into Other Forms

You eventually need to consider how to work with forms of data that you require for your analysis that simply defy easy storage using existing methods. Because you can find nearly constant development in this area, you can safely assume that other data scientists face the same issues as you do.

One of the more interesting newer entries into this field is object storage of the sort provided by Amazon Web Services (AWS) Simple Storage Service (S3) (<https://aws.amazon.com/what-is-cloud-object-storage/>). The methodology seems incredibly messy at first because you simply throw objects into what amounts to an electronic bucket. The data doesn't even appear neatly stacked on shelves. Yet, this storage method offers fast and easy storage of items that just don't fit anywhere else and seemingly defy organization.

**TIP**

Image files can prove especially difficult to deal with because not only are they stored in a hierarchical manner but the images themselves can present problems. The article at <https://realpython.com/storing-images-in-python/> offers some interesting ways to work with image files that you want to use as part of an analysis. Besides the usual disk storage technique (using the Pillow library), this article tells you about these techniques:

- Lightning Memory-Mapped Databases (LMDB)
- Hierarchical Data Format (HDF5)

Table of contents collapsed